COURSE OVERVIEW

Build a pickup coordination system

Three questions at a university volunteer desk



What is offered? Who will collect it? What actually happened?

The scenario: you run the university pickup desk

A donor contacts you

“Five bread containers and two rice containers are available.”
You need units and a reviewed arrangement.

Volunteers need a plan

Choose quantities from several batches.
Avoid promising the same stock twice.

A partner awaits delivery

The coordinator needs to know what is reserved, collected and delivered.

Your job: give the coordinator one reliable place to plan, track and explain pickups.

The destination: one coordination dashboard

OFFERED	RESERVED	COLLECTED	DELIVERED
Week-8	Week-8	Week-8	Week-8
94	60	42	29

CONCEPT VIEW · Python + SQLite · Web UI optional

Week-9 / Week-10 · improve + hand over

Available offers

Week-5 / Week-6

Batch	Left
B01 · Bread packs	9
B02 · Rice meal sets	2
B03 · Vegetable trays	2

Saved IDs + units

Plan a pickup

Week-1 to Week-4

New draft: B01 × 1, B02 × 1
2 containers / 7 planned portions

Review: Yes → standard

Reserve all lines — Week-7

Pickup status

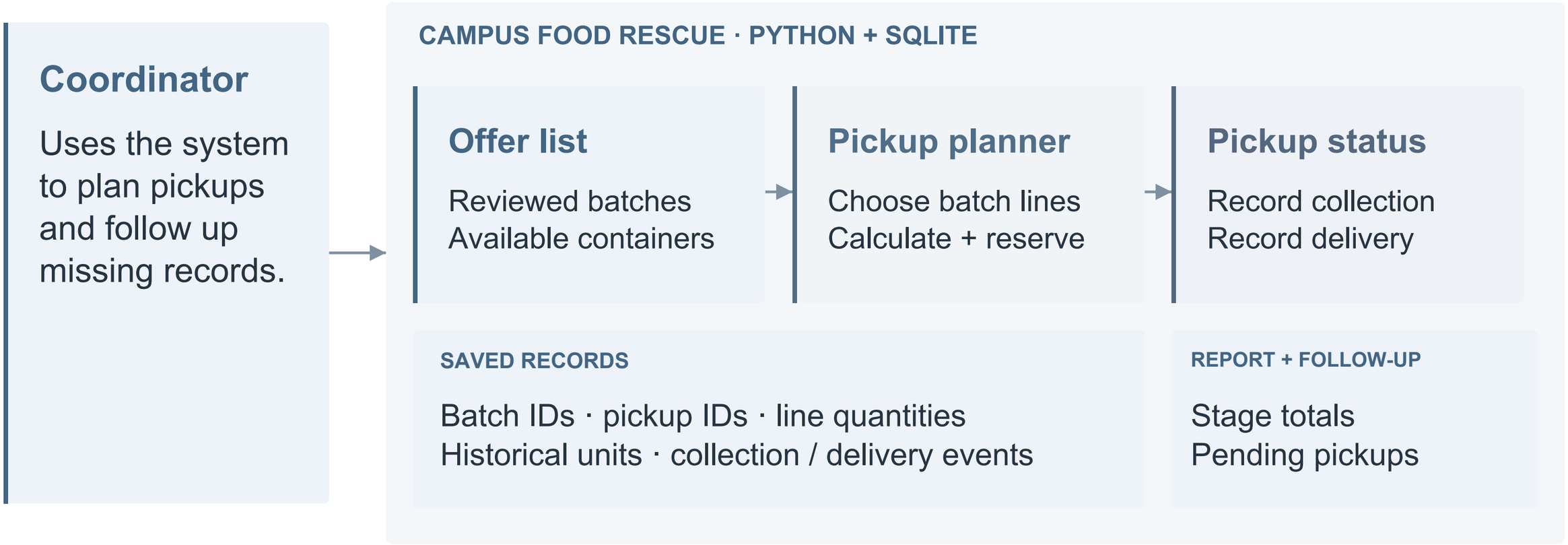
Week-6 to Week-8

P01	Delivered
P03	Delivered
P05	Collected
P06	Reserved only

Follow up: P06 awaits collection

All amounts are declared planning portions. Stage totals overlap; do not add them.

What are we building? A pickup coordination system



The coordinator operates it; Python applies rules; SQLite keeps the records.

What enters the system—and what comes out?

Inputs

Batch ID, declared unit and available count
Requested quantities
Human review and observed events

Actions

Calculate a plan
Accept or refuse a reservation
Record collection and delivery
Query and follow up

Outputs

Pickup summary and dispatch label
Remaining availability
Status by pickup ID
Reports and pending work

A useful result answers a coordinator question and can be traced to saved records.

A concrete operation: plan and reserve one pickup

CONCEPT VIEW · PICKUP DESK

1 Choose from reviewed offers

Batch	Available	Request
DEMO bread	5 containers	3 containers
DEMO rice	2 containers	1 container

Units: bread 2 / container; rice 5 / container

2 Inspect the proposed pickup

Bread: 3 × 2 = 6 planned portions
Rice: 1 × 5 = 5 planned portions
Total: 4 containers / 11 planned portions

Coordinator review recorded: Yes

Reserve the complete pickup

Illustrative interaction; the course implements these operations with Python and SQLite.

Follow the same pickup after each action

Action	What the record says	What the coordinator sees
Plan only	4 containers / 11 planned portions	Availability is still bread 5, rice 2
Reserve pickup	Accepted; 11 portions reserved	Availability becomes bread 2, rice 1
Record collection	11 portions collected	Delivery is still unrecorded
Record delivery	11 portions delivered	The pickup has delivery evidence

Collection and delivery update status; they do not subtract the stock a second time.

Who supplies the judgment and who performs the work?

Donor + coordinator

Clarify container contents.
Approve suitability, partner arrangements and the collection window.

Python + SQLite

Calculate and validate.
Protect availability.
Save records and query evidence.

Volunteers + partner

Carry out collection and delivery.
Provide the observations that the coordinator records.

The system coordinates and records the work; people make arrangements and move the food.

The team needs more than a message

Unclear quantity

“Four boxes available.”
What is inside each box?
Which unit are we counting?

Conflicting actions

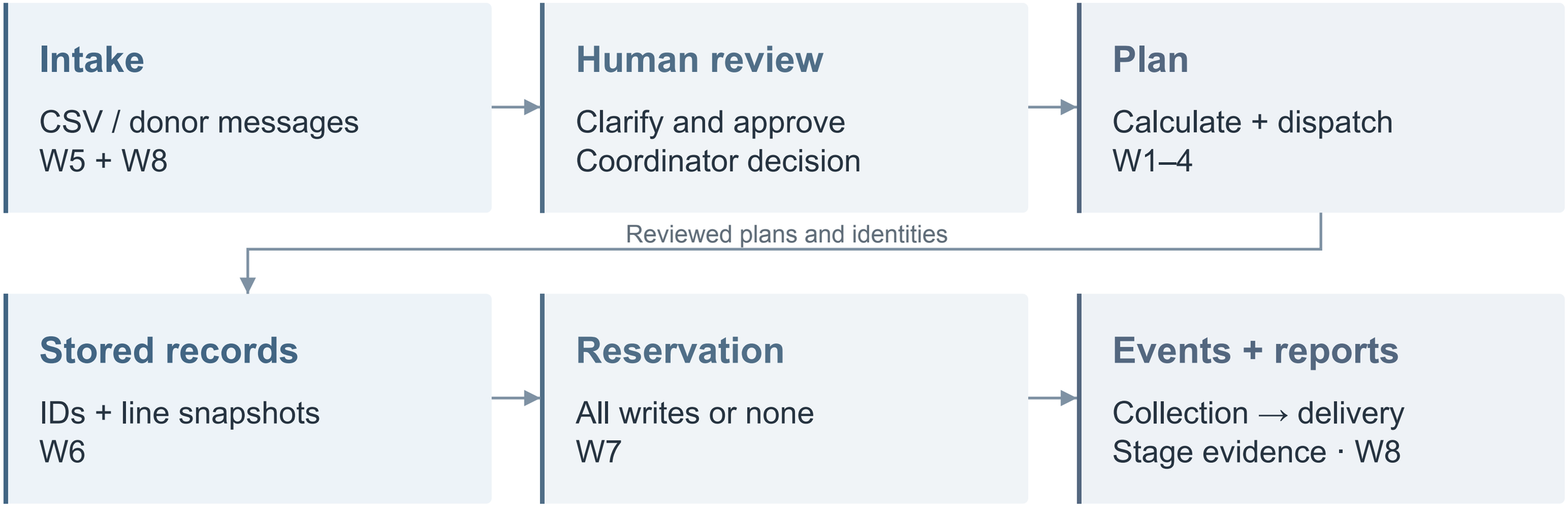
Two volunteers try to reserve
the same containers.
Which request can succeed?

Uncertain outcome

An accepted pickup is reported.
Was it collected?
Was delivery recorded?

You are building the information system that makes these decisions traceable.

System map: how information moves



W9: improve one component → W10: verify and hand it over

This is the workflow; the next pages show the order in which we learn its parts.

Know what each record can establish

Record	What it means	What it does not prove
Donor proposal	Candidate to clarify	Reservable availability
Reviewed offer	Reviewed opening stock	Collection or delivery
Pickup plan	Proposed lines + estimate	Reserved containers
Reservation	Accepted stock commitment	Physical movement
Collection event	Collection recorded	Partner delivery
Delivery event	Delivery after collection	Unique people served

Name the unit, source and lifecycle stage behind every number.

Break the work into connected parts

1

Values and rules

Name inputs.
Calculate totals.
State a decision.

2

Validated records

Check outside data.
Keep identity and units.
Save useful facts.

3

Reliable updates

Group related writes.
Reject incomplete actions.
Inspect failure state.

4

Evidence and change

Record events.
Reconcile reports.
Test an improvement.

Each part needs a clear input, output and check before another part can rely on it.

Learning order: three phases

1

Weeks 1–4

Plan and decide
Values, branches, loops
and reusable functions.

2

Weeks 5–8

Keep records reliable
Validate, save, reserve
and explain stage reports.

3

Weeks 9–10

Improve and hand over
Implement one change,
verify it and explain it.

The learning order starts small; each weekly component prepares a later capability.

Week 01 · One-batch planning note

WEEK 01

One-batch planning note

Turn a declared count into a readable estimate.

BUILD WITH

Variables · types · arithmetic · strings · .py execution

SHOW IT WORKS

Inputs, labelled total and one question about units.

CONNECTS TO

Week 2 uses a total in a review-first decision.

Build one useful component, explain its behavior and keep evidence.

Week 02 · Review-first dispatch

WEEK 02

Review-first dispatch

A total alone cannot decide whether a request may proceed.

BUILD WITH

Comparisons · Boolean logic · if/elif/else · boundary tests

SHOW IT WORKS

A six-row decision table and checked labels.

CONNECTS TO

Week 3 calculates the total from several batch lines.

Build one useful component, explain its behavior and keep evidence.

Week 03 · Multi-batch aggregation

WEEK 03

Multi-batch aggregation

Count lines, containers and portions without confusing them.

BUILD WITH

Lists · dictionaries · for loops · accumulators

SHOW IT WORKS

Three labelled totals, a hand trace and empty/split tests.

CONNECTS TO

Week 4 puts a reusable rule behind a function boundary.

Build one useful component, explain its behavior and keep evidence.

Week 04 · Functions and Midterm

WEEK 04

Reusable policy and tests

Let a notebook and a saved program use an explained rule.

BUILD WITH

Functions · return · loop totals · assertions · debugging

SHOW IT WORKS

Canonical Midterm function, four common tests and reproducible ZIP.

CONNECTS TO

Week 5 adds checked external inputs to reusable logic.

Build one useful component, explain its behavior and keep evidence.

Week 05 · Proposal importer and audit

WEEK 05

Proposal importer and audit

Explain malformed, repeated and conflicting donor claims.

BUILD WITH

CSV · conversion · validation · exceptions · CLI arguments

SHOW IT WORKS

A disposition, reason and source identity for every input row.

CONNECTS TO

Week 6 saves identifiable records and historical units.

Build one useful component, explain its behavior and keep evidence.

Week 06 · Persistent plan records

WEEK 06

Persistent plan records

Retrieve the same plan after closing the program.

BUILD WITH

Objects/methods · SQLite · IDs · relationships · parameters

SHOW IT WORKS

A saved plan, line-unit snapshots and fresh-connection lookup.

CONNECTS TO

Week 7 groups operational record changes in one transaction.

Build one useful component, explain its behavior and keep evidence.

Week 07 · Atomic reservation

WEEK 07

Atomic reservation

Avoid half a reservation when a later batch cannot be filled.

BUILD WITH

Transactions · guarded UPDATE · exceptions · snapshots · tests

SHOW IT WORKS

Complete success, or unchanged state after a required failure.

CONNECTS TO

Week 8 reports accepted reservations and explicit events.

Build one useful component, explain its behavior and keep evidence.

Week 08 · Stage reports and intake review

WEEK 08

Stage reports and intake review

Explain stage totals and turn uncertain messages into follow-up.

BUILD WITH

SQL aggregation · reconciliation · HTTP · JSON · validation

SHOW IT WORKS

Stage evidence, six intake outcomes and a project proposal.

CONNECTS TO

Week 9 implements one need with two agreed acceptance cases.

Build one useful component, explain its behavior and keep evidence.

Week 09 · Student-designed improvement

WEEK 09

Student-designed improvement

Help one stakeholder make a clearer decision.

BUILD WITH

Scoped functions/queries · acceptance · regression · peer review

SHOW IT WORKS

A working change, peer finding, revision and retest.

CONNECTS TO

Week 10 makes the feature reproducible by another operator.

Build one useful component, explain its behavior and keep evidence.

Week 10 · Verified handoff

WEEK 10

Verified handoff

Let another volunteer run the system without your memory.

BUILD WITH

Clean starts · run commands · recovery · independent verification

SHOW IT WORKS

A reproducible guide, operator demo and individual explanation.

CONNECTS TO

The next operator can trace the result and its limitations.

Build one useful component, explain its behavior and keep evidence.

Every week: learn Python, practise, then build

Session 1 · Python

Start with the programming topic.
Read syntax and trace short code
in the weekly notebook.

Session 2 · Practice

Use the same weekly notebook.
Predict, run and change examples.
Explain a result with a partner.

Session 3 · Project

Use a short design briefing.
Implement the weekly component.
Discuss, check and keep evidence.

Follow Start → Sessions 1–3 → Finish in one weekly notebook. Basics is optional lookup.

Use the right material for the job

Python Basics

Optional syntax lookup
19 short topics
Small examples and cautions

Weekly notebook

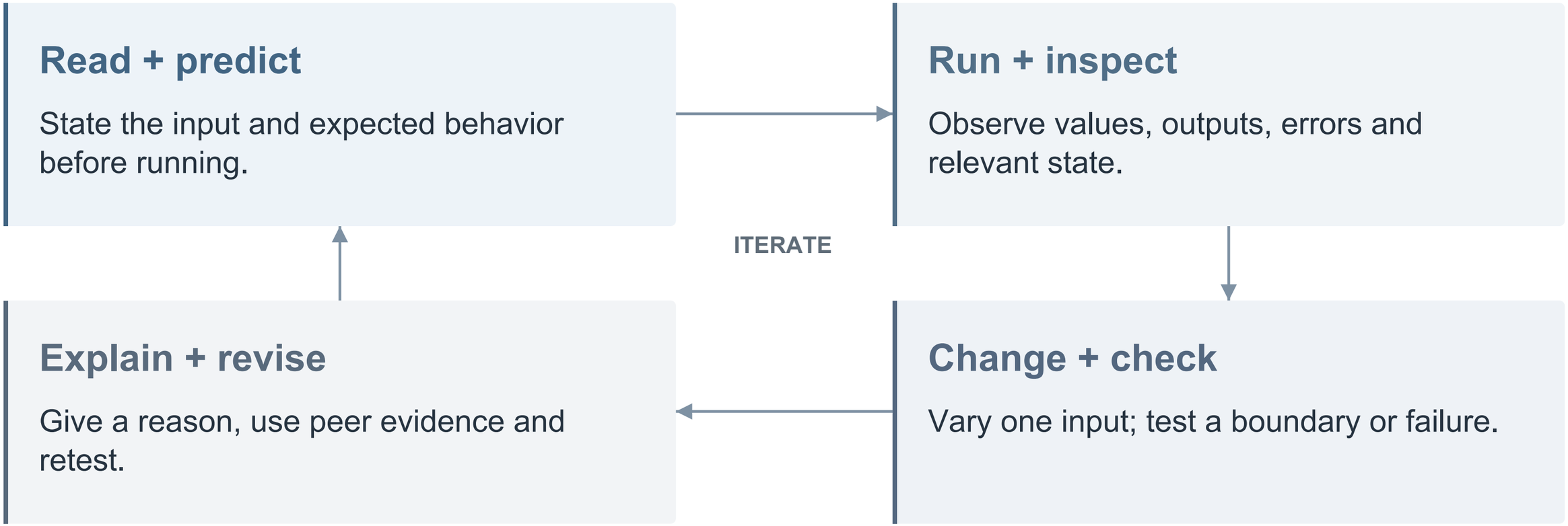
Full explanations and examples
Python practice + project tasks
Start → Sessions 1–3 → Finish

Assignment brief

Detailed task and expected evidence
A small example
Midterm/final guides separately

Each week has a starting point; an unfinished earlier notebook does not block you.

Make learning visible in a small cycle



Save the required evidence and make sure you can explain it; optional extensions can wait.

A service response still needs verification

Communication

Week 8 uses actual local HTTP
with authored replay responses.
It is not live model inference.

Meaning

Compare the candidate with the original donor message.
Check fields, units and omissions.

Human judgment

Keep uncertain claims in a queue.
A cancellation is not a new offer.
A coordinator approves arrangements.

Suggested code and automated output must be inspected, tested and explained.

Your final project: one small, verified improvement

Agree a need

Pitch in Week 8.
Choose a stakeholder and scope.
Freeze two acceptance cases.

Build and verify

Use the supplied baseline.
Keep an unaffected case and a boundary/failure case.
Record peer feedback and retest.

Demonstrate and hand over

Week 10: let a peer operate.
Provide reset/run instructions.
Explain your work individually.

A reliable Python or SQL change is enough; a web app or paid AI integration is not required.

Six short research tasks support the project

Research / topic	Assigned	Shared
1 · Goals + systems	Week 1	Week 2
2 · Functions + tests	Week 2	Week 3
3 · Input validation	After Week 4 midterm	Week 5
4 · Persistent records	Week 5	Week 6
5 · Failure handling	Week 6	Week 7
6 · Output checks	Week 7	Start of Week 8

Proposed 5% each · sources, example and your reasoning · presentation is not required.

Assessment rewards building, checking and explaining

Component	Proposed weight	Evidence
Participation	10%	Attendance and reflection
Six research tasks	30%	Sources, example, reasoning
Individual midterm	30%	Function, tests, ZIP, explanation
Final project	30%	Change, verification, handoff

The instructor and university will confirm the plan before grading begins.

Get started and keep questions useful

Open the package

Use learning_materials/ and keep its support folders together.
Follow the setup guide.

Choose an environment

Local Python/Jupyter or Colab.
Upload the support ZIP in Colab.
Save your work and a backup.

Ask with evidence

Week + small input + expectation
+ observed result + question.
Use the announced class route.

Use the course home for available weeks, links, assignment briefs and setup instructions.

Further reading in English

1. ## Python Tutorial — classes and exceptions

   https://docs.python.org/3/tutorial/

2. ## Python sqlite3 — database connections and transactions

   https://docs.python.org/3/library/sqlite3.html

3. ## Python unittest — organizing and running tests

   https://docs.python.org/3/library/unittest.html

Read one relevant section, then bring an example and a question.